

Tabla de contenidos

1. Terminal
2. Comandos
3. Ejecución de Python en Terminal

Terminal

1. ¿Qué es la terminal o consola?

Una terminal o consola es una de las formas para comunicarnos con un computador. Esta nos provee de una interfaz de texto para darle instrucciones o comandos al computador. Por ejemplo, ejecutar programas instalados previamente como Python y Jupyter Notebook, navegar por el sistema de archivos del computador, explorar qué archivos hay en determinadas carpetas, entre otros.

Su funcionamiento se basa en que el usuario escribe comandos junto con sus parámetros para que el sistema operativo entregue una respuesta. A continuación se muestra una sesión de terminal en Lubuntu (el entorno estándar del curso), donde se ejecutó el comando `cd` para acceder al escritorio del computador y `ls` para saber qué archivos y carpetas hay en el escritorio:

```
pedro@lubuntu:~$ cd Escritorio
pedro@lubuntu:~/Escritorio$ ls
contenidos  Syllabus  tarea-01
```

Para efectos de este curso, las dos principales ventajas de utilizar la terminal son:

1. Poder tener varias terminales abiertas y cada una utilizarla para diferentes propósitos. Por ejemplo, **ejecutar diferentes archivos Python de forma simultánea**.
2. Ser capaz de saber siempre desde qué carpeta estamos ejecutando un archivo. Lo cual nos reducirá la posibilidad de tener errores relacionados a **rutas relativas**.

2. ¿Cómo abrir la terminal?

- **Lubuntu (el entorno estándar del curso):** puedes usar la combinación de teclas `Ctrl + Alt + T`, o abrir el menú de aplicaciones y buscar "QTerminal" (Menú ▢ Herramientas del sistema ▢ QTerminal).
- **Otras distribuciones de Linux:** la combinación `Ctrl + Alt + T` suele abrir la terminal por defecto del sistema.
- **Windows:** en la barra de inicio, puedes buscar "cmd" y abrir el programa ("Command Prompt"). Aunque también recomendamos descargar desde la tienda la aplicación "Windows Terminal" y usarla como terminal.
- **macOS:** en Finder, abre la carpeta /Aplicaciones/Utilidades y haz doble clic en "Terminal".

Comandos

Los comandos corresponden al mecanismo donde el usuario se puede comunicar con el computador y solicitar que haga ciertas acciones. Para este curso, los comandos más importantes a conocer son 6: `pwd` (en Windows, `cd` sin argumentos), `cd`, `ls` (o `dir` en Windows), `mkdir`, `touch` (o `echo` en Windows) y `python3` (en Windows, `python`).

1. Formato del comando

El formato para todo comando es el siguiente:

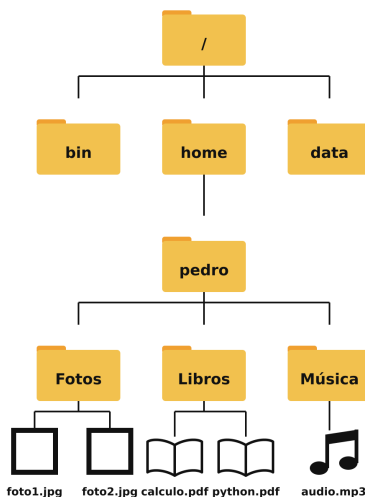
```
comando argumentos* -opciones*
```

- **comando:** corresponde al nombre del comando a llamar. Por ejemplo, `pwd`, `cd`, `python3`, `pip`.
- **argumentos*:** corresponde a los parámetros que el comando va a utilizar para su ejecución. Dependiendo del comando, estos argumentos pueden ser 0, 1, 2, etc. Algunos casos son:
 1. `pwd` es un comando que no necesita argumentos, solo escribimos dicho comando y funcionará.
 2. `python3` es un comando que cambia su funcionamiento según la cantidad de argumentos. Si se ejecuta sin ningún argumento (`python3`), se abrirá la consola de Python. Si se ejecuta con 1 argumento que corresponde a un archivo `.py` (`python3 ejemplo3.py`), se ejecutará el archivo correspondiente. En Windows, el comando se llama `python`.
 3. `pip` es un comando que interactúa con el gestor de librerías de Python, y este tiene muchos argumentos posibles. Para ver las librerías instaladas, se necesita 1 argumento, `list`, para su ejecución: `pip list`. Para instalar una librería, se necesitan 2 argumentos, `install nombre_librería`, para su ejecución: `pip install pyqt6`. En Ubuntu, si el comando `pip` no está disponible, puedes instalarlo con `sudo apt install python3-pip` (o usar `python3 -m pip` si el módulo ya está instalado).
- **-opciones:** también conocidos como *flags*, es la forma de especificar opciones durante la ejecución del comando. Cada opción se escribe como `-nombre_de_la_opción parámetro*` en donde el parámetro puede ser opcional según la opción que estemos llamando. Por ejemplo:
 1. El comando `ls` lista los archivos y carpetas del directorio actual, pero si se ejecuta como `ls -a` (de *all*), también mostrará los archivos ocultos.
 2. Para instalar una librería de Python se puede hacer `pip install pyqt6`, pero también se puede hacer `pip install pyqt6 -q`. El `-q` se encarga de personalizar la ejecución, en este caso, hacer la instalación "quiet", es decir, sin imprimir tantas cosas en consola.

✉ **Nota para Ubuntu/Lubuntu:** instalar librerías con `pip` fuera de un entorno virtual puede fallar con el error `externally-managed-environment`. No necesitarás instalar librerías por ahora; cuando el curso lo requiera, te indicaremos el procedimiento adecuado.

2. Uso de comandos

Para ejemplificar cada comando que veremos a continuación, imaginemos que tenemos la siguiente estructura de carpetas y archivos:



Linux y macOS Tanto Linux (incluido Lubuntu) como macOS funcionan con facilidades estándar de Unix en sus terminales. Esto nos permite utilizar los mismos comandos para ambos sistemas operativos: `pwd`,

cd, ls, mkdir y touch.

En Linux, las carpetas personales de los usuarios están dentro de `/home`; en macOS, dentro de `/Users`. En los ejemplos siguientes usaremos la estructura de la figura anterior, con un usuario `pedro` cuya carpeta personal es `/home/pedro`. En la terminal, el símbolo `~` que aparece en el *prompt* es una abreviación de la carpeta personal del usuario.

1. Ver directorio actual (pwd) De las siglas en inglés **print working directory**, es el comando que nos retorna la **ruta absoluta** para llegar a nuestra posición desde la terminal. De forma informal, podemos decir que este comando nos responde a la pregunta: ¿Dónde estamos?

```
pedro@lubuntu:~$ pwd
/home/pedro
```

En este caso, nuestra terminal está ubicada dentro de la carpeta `pedro` que a su vez está dentro de la carpeta `home`, la cual está dentro de la carpeta raíz.

2. Cambiar directorio actual (cd) Del inglés **change directory**, este comando nos permite movernos en el sistema de archivos para acceder a diferentes directorios/carpetas. Para esto, es necesario agregar una **ruta absoluta** o **relativa** como argumento.

```
pedro@lubuntu:~$ pwd
/home/pedro
pedro@lubuntu:~$ cd Libros
pedro@lubuntu:~/Libros$ pwd
/home/pedro/Libros
```

Ahora, desde la carpeta `pedro` nos movimos a la carpeta `Libros`. También es posible salir de una carpeta utilizando el argumento `..`, que representa a la carpeta superior:

```
pedro@lubuntu:~/Libros$ cd ..
pedro@lubuntu:~$ pwd
/home/pedro
pedro@lubuntu:~$ cd Música
pedro@lubuntu:~/Música$ cd ../../../../data
pedro@lubuntu:/data$ pwd
/data
```

En el último comando encadenamos tres `..`: desde `/home/pedro/Música` subimos a `/home/pedro`, luego a `/home`, luego a la raíz `/`, y desde ahí entramos a `data`.

3. Listar contenido de un directorio (ls) Del inglés **list**, este comando nos entrega un detalle de los archivos y carpetas presentes en un determinado directorio. De forma informal, podemos decir que este comando nos responde a la pregunta: ¿Qué tenemos en la carpeta?

```
pedro@lubuntu:~$ pwd
/home/pedro
pedro@lubuntu:~$ ls
Fotos  Libros  Música
pedro@lubuntu:~$ cd Libros
```

```
pedro@lubuntu:~/Libros$ ls
calculo.pdf  python.pdf
```

4. Crear un directorio (mkdir) Del inglés **make directory**, este comando crea una nueva carpeta o subcarpeta con el nombre indicado como argumento. También podemos entregarle el *path* con la dirección hacia la nueva carpeta.

```
pedro@lubuntu:~$ ls
Fotos  Libros  Música
pedro@lubuntu:~$ mkdir Codigos
pedro@lubuntu:~$ ls
Codigos  Fotos  Libros  Música
pedro@lubuntu:~$ mkdir /home/pedro/Apuntes
pedro@lubuntu:~$ ls
Apuntes  Codigos  Fotos  Libros  Música
```

5. Crear un archivo vacío (touch) Este comando "toca" el archivo para comprobar si existe o no en esa ruta. Si un archivo no existe en el directorio indicado, crea un archivo vacío con el nombre y la extensión. Si el archivo existe, actualiza instantáneamente sus fechas de último acceso y de última modificación al momento de ejecución del comando. Su nombre "touch" en español es "tocar", como si estuvieras comprobando si está ahí o no con un palito. De forma informal, podemos decir que este comando nos responde a la pregunta: ¿Existes en este momento?

```
pedro@lubuntu:~$ touch main.py
```

Al ejecutar este comando, la terminal no emitirá ningún mensaje. Si el archivo pedido main.py no existe, podrás comprobar con `ls` que ha sido creado y está vacío.

¡Te invitamos a probar estos 5 comandos (pwd, cd, ls, mkdir y touch) en tu terminal para familiarizarte con su uso! Utilizarás mucho estos comandos en este curso.

Windows Si usas Windows en tu computador personal, la terminal clásica (cmd) usa otros comandos: `cd`, `dir`, `mkdir` y `echo`. En Windows, las carpetas personales están dentro de `C:\Users`, por ejemplo `C:\Users\Pedro`. Usaremos la estructura de la figura anterior, que en Windows correspondería a `C:\Users\Pedro` y `C:\data`.

1. Ver directorio actual (cd) Del inglés **change directory**, este comando nos permite movernos en el sistema de archivos para acceder a diferentes directorios/carpetas. Cuando ejecutamos este comando sin ningún argumento, nos retorna la posición actual como una **ruta absoluta**:

```
C:\Users\Pedro> cd
C:\Users\Pedro
```

En este caso, nuestra terminal está ubicada dentro de la carpeta `Pedro` que a su vez está dentro de la carpeta `Users`, la cual está dentro de la carpeta raíz `C:\`.

2. Cambiar directorio actual (cd) En caso de que se agregue una ruta (absoluta o relativa) al comando, este nos permitirá cambiar nuestro directorio:

```
C:\Users\Pedro> cd Libros
C:\Users\Pedro\Libros> cd
C:\Users\Pedro\Libros
```

Ahora, desde la carpeta Pedro nos movimos a la carpeta Libros. También es posible subir a la carpeta superior utilizando el argumento ...:

```
C:\Users\Pedro\Libros> cd ..
C:\Users\Pedro> cd Música
C:\Users\Pedro\Música> cd ..
C:\Users\Pedro> cd ../../data
C:\data> cd
C:\data
```

3. Listar contenido de un directorio (dir) Del inglés **directory**, este comando nos entrega un detalle de los archivos y carpetas presentes en un determinado directorio. De forma informal, podemos decir que este comando nos responde a la pregunta: ¿Qué tenemos en la carpeta?

```
C:\Users\Pedro> dir
... (texto omitido para este ejemplo)

Directorio de C:\Users\Pedro
02-08-2026  12:29    <DIR>        .
02-08-2026  12:29    <DIR>        ..
19-07-2026  18:45    <DIR>        Fotos
19-07-2026  18:45    <DIR>        Libros
19-07-2026  18:45    <DIR>        Música
... (texto omitido para este ejemplo)
```

```
C:\Users\Pedro> cd Libros
C:\Users\Pedro\Libros> dir
... (texto omitido para este ejemplo)

Directorio de C:\Users\Pedro\Libros
02-08-2026  12:29    <DIR>        .
02-08-2026  12:29    <DIR>        ..
02-08-2026  12:29                11.960 calculo.pdf
02-08-2026  11:55                16.119 python.pdf
... (texto omitido para este ejemplo)
```

4. Crear un directorio nuevo (mkdir) Del inglés **make directory**, este comando crea una nueva carpeta o subcarpeta con el nombre indicado como argumento. También podemos entregarle el *path* con la dirección hacia la nueva carpeta:

```
C:\Users\Pedro> mkdir Codigos
C:\Users\Pedro> dir
... (texto omitido para este ejemplo)

Directorio de C:\Users\Pedro
07-08-2026  10:00    <DIR>        .
```

```

07-08-2026 10:00 <DIR>      ..
07-08-2026 10:00 <DIR>      Codigos
19-07-2026 18:45 <DIR>      Fotos
19-07-2026 18:45 <DIR>      Libros
19-07-2026 18:45 <DIR>      Música
... (texto omitido para este ejemplo)

```

5. Crear un archivo nuevo (echo) Este comando permite mostrar un mensaje de salida en la terminal, lo más parecido a un print de Python pero en consola. En este caso, lo usaremos para redirigir esa salida a un archivo. La sintaxis es: `echo "TEXTO" > NOMBRE_ARCHIVO`, donde `echo` es el nombre del comando, `"TEXTO"` es el texto de salida entre comillas simples o dobles, y `NOMBRE_ARCHIVO` será el nombre de nuestro nuevo archivo considerando siempre la extensión (.txt, .py).

```
C:\Users\Pedro> echo "" > nuevo.py
```

Nota: el archivo creado de esta forma no queda completamente vacío (contiene el texto indicado, en este caso ""). Para crear un archivo realmente vacío en cmd puedes usar `type nul > nuevo.py`.

¡Te invitamos a probar estos 4 comandos (cd, dir, mkdir y echo) en tu terminal para familiarizarte con su uso! Utilizarás mucho estos comandos en este curso.

Ejecución de Python en Terminal

La mayoría de las *IDEs* para programar en Python ofrecen la opción de ejecutar esos archivos con solo oprimir un botón. No obstante, en muchos casos, su interfaz no muestra claramente desde dónde se está ejecutando el archivo, es decir, no se sabe tan fácilmente el `pwd` desde donde se ejecutó el archivo Python. Esto en varias ocasiones genera **problemas** con la apertura de archivos y también con los `imports` o cuando utilizamos **rutas relativas** en el código.

Adicionalmente, estas *IDEs* vienen por defecto con la configuración para ejecutar **1 archivo a la vez**. Esto provoca que para actividades donde el estudiantado requiere ejecutar **2 o más archivos a la vez**, no se logre.

En vista de las dificultades presentadas anteriormente, a continuación se explica cómo poder ejecutar archivos de Python en terminal. Esto nos dará la ventaja de:

1. Saber claramente nuestra ubicación en el sistema de archivos.
2. Evitar problemas con la apertura de archivos, los `imports` y con las **rutas relativas** dentro del código.
3. Poder ejecutar más de 1 archivo simultáneamente.

1. Abrir Python

Desde la terminal, podemos escribir `python3` (en Windows, el comando es `python`) y podremos acceder a la consola interactiva de Python, donde podremos ejecutar código de Python línea a línea. Por ejemplo, si necesitamos hacer una operación rápida (y no queremos usar nuestra calculadora):

```

pedro@lubuntu:~$ python3
Python 3.12.3 (...) on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 2 + 2
4

```

Para salir de la consola de Python y volver a la terminal, basta ejecutar `exit()`.

2. Ejecutar script de Python

Otra ventaja de usar la terminal es poder ejecutar nuestros archivos Python sin recurrir a alguna *IDE* (como Visual Studio Code o Sublime Text). Para esto, solo basta agregar como argumento el nombre del archivo a ejecutar.

En el siguiente caso, vamos a acceder a la carpeta `semana-02` que está dentro de `contenidos` en el escritorio, y vamos a ejecutar el archivo `ejemplo3.py` que se encuentra junto a estos apuntes:

```
pedro@lubuntu:~$ cd Escritorio/contenidos/semana-02
pedro@lubuntu:~/Escritorio/contenidos/semana-02$ python3 ejemplo3.py
Hello World
```

También es posible ejecutar un archivo que no esté dentro de nuestro directorio actual. Para esto, es necesario escribir la **ruta absoluta** o **relativa** a dicho archivo. En el siguiente ejemplo se ejecutó el mismo archivo, pero en el primer caso se utilizó una **ruta relativa** para acceder a él y en el segundo caso se utilizó una **ruta absoluta**:

```
pedro@lubuntu:~$ python3 Escritorio/contenidos/semana-02/ejemplo3.py
Hello World
pedro@lubuntu:~$ python3 /home/pedro/Escritorio/contenidos/semana-02/ejemplo3.py
Hello World
```

❗ **Importante** ❗: esto hace que `ejemplo3.py` crea que está siendo ejecutado desde la ruta actual de la consola, que es distinta a la carpeta donde se encuentra el archivo. Lo cual puede generar un error, como veremos a continuación.

3. Errores comunes en Python con la terminal

1. Archivo no existe

Es posible que el archivo a ejecutar o la ruta a dicho archivo no estén bien escritos o simplemente no exista dicho archivo. En ese caso Python retornará un error indicando que el archivo no existe:

```
pedro@lubuntu:~$ python3 Escritorio/contenidos/semana-02/no_existe.py
python3: can't open file '/home/pedro/Escritorio/contenidos/semana-02/no_existe.py': [Errno 2] No s
```

1. Conflictos con rutas relativas

Es posible que el archivo a ejecutar ocupe rutas relativas y esto genere conflictos según donde ejecutamos el archivo. Por ejemplo, junto a estos contenidos hay un directorio llamado `carpeta_auxiliar`. Dentro de este existen 2 archivos: `hello_world.txt` y `main.py`. El contenido de este segundo archivo es

```
import os

if __name__ == '__main__':
    print("Me estoy ejecutando desde:")
    path = os.getcwd()
    print(path)
    content = open("hello_world.txt").read()
    print(content)
```

Es decir, este nuevo archivo nos indica desde dónde se está ejecutando el Python y luego abre `hello world.txt` para imprimir su contenido. Dado que solo está escrito el nombre del archivo, `main.py` asume que `hello world.txt` está en el mismo directorio desde donde se ejecutará `main.py`.

Ahora, desde la terminal, vamos a posicionarnos en la carpeta de los contenidos de la semana 2 y vamos a intentar ejecutar el `main.py`:

```
pedro@lubuntu:~/Escritorio/contenidos/semana-02$ python3 carpeta_auxiliar/main.py
Me estoy ejecutando desde:
/home/pedro/Escritorio/contenidos/semana-02
Traceback (most recent call last):
  File "/home/pedro/Escritorio/contenidos/semana-02/carpeta_auxiliar/main.py", line 8, in <module>
    content = open("hello world.txt").read()
              ~~~~~^
FileNotFoundError: [Errno 2] No such file or directory: 'hello world.txt'
```

Esto ocurrió porque nosotros ejecutamos `main.py` desde el directorio `/home/pedro/Escritorio/contenidos/semana-02`. En dicho directorio **no existe** `hello world.txt`.

Ahora, utilizaremos `cd` para acceder al directorio `carpeta_auxiliar` y volvemos a ejecutar `main.py`:

```
pedro@lubuntu:~/Escritorio/contenidos/semana-02$ cd carpeta_auxiliar
pedro@lubuntu:~/Escritorio/contenidos/semana-02/carpeta_auxiliar$ python3 main.py
Me estoy ejecutando desde:
/home/pedro/Escritorio/contenidos/semana-02/carpeta_auxiliar
hello world
```

Ahora, sí funciona porque efectivamente en `/home/pedro/Escritorio/contenidos/semana-02/carpeta_auxiliar` sí existe el archivo `hello world.txt`.

Importante: siempre tengan en mente desde qué carpeta se ejecutarán sus archivos Python para no tener problema con las rutas relativas.